Improving Numerical Determinism in LLM Inference

Deep Learning (CMP030-L043-0) Part 2 Assessment Report

Collins Chikaodi Collins A00070784

Word Count: 2965

Table of Contents

Artefact Overview and Implementation....1
1.1 System Overview.... 1
1.2 Relationship to Part 1 Design....2
1.3 System Architecture and Components....3
1.4 Implementation Details....3
1.5 Deviation From Original Purpose.... 4
2. Data and Methodology....4
2.1 Development Environment....4
2.2 Model Selection (TinyLlama)....5
2.3 Experimental Methodology....5
2.4 Deterministic inference Process....6
2.5 Design choice....6
3.Experimentation and Evaluation.... 8
3.1 Experimental Setup.... 8
3.2 Evaluation Metrics.... 8
3.3 Baseline Evaluation....9
3.4 Precision Evaluation....10
3.5 Result Interpretation....10
4. Performance, Robustness and Applicability....11
4.1 Performance Analysis....11
4.2 Numerical Determinism Analysis....12
4.3 Generalisation.... 12
4.4 Real-World Applicability.... 12
5. System Engineering Considerations, Reflection and Limitations.... 13
5.1 Reproducibility....13
5.2 Scalability.... 14
5.3 Deployment Considerations.... 14
5.4 Github Repository....15
5.5 Limitations.... 15
5.6 Ethical Consideration.... 15
5.7 Reflection and Future Work....16
Reference....17
APPENDIX.... 1
Appendix A: Github Repository.... 1
Appendix B: Video Demonstration.... 1
Appendix C: Experimental Environment....1

Artefact Overview and Implementation

1.1 System Overview

Large Language Models (LLMs) have become fundamental to modern Artificial Intelligence, enabling applications such as conversational assistants, software development, scientific research, and intelligent decision-support systems. As these models are increasingly deployed in real-world environments, ensuring that they produce consistent and reproducible outputs has become an important engineering challenge. Although deterministic decoding aims to generate identical responses for identical prompts, numerical variations introduced during floating-point computations can still affect inference behaviour. Such inconsistencies reduce reproducibility and present challenges for testing, debugging, and deploying reliable AI systems.

This project presents an artefact that investigates numerical determinism during LLM inference using the TinyLlama-1.1B-Chat model. The implementation evaluates inference consistency by measuring inference time, GPU memory utilisation, response length, and generated outputs under deterministic decoding conditions. Developed using Python, PyTorch, and the Hugging Face Transformers library within a Kaggle Notebook environment, the artefact provides a reproducible workflow for analysing numerical determinism. Experimental results are exported in CSV and Excel formats and maintained alongside the implementation in a GitHub repository. **Figure 1** illustrates the overall architecture of the proposed system.

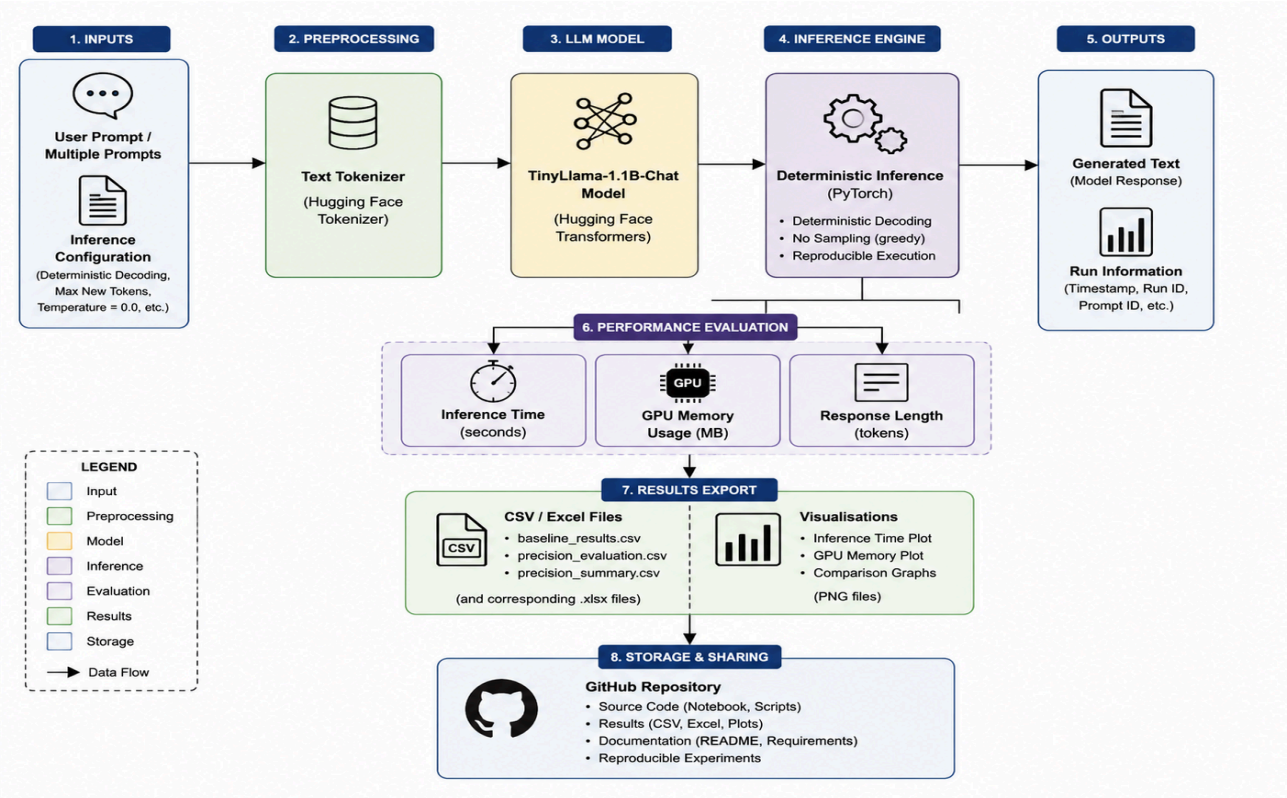

Figure 1 illustrates the overall architecture of the proposed system .

1.2 Relationship to Part 1 Design

The implementation directly builds upon the design proposed in Part 1, which identified numerical non-determinism during Large Language Model inference as a challenge affecting reproducibility. The proposed solution was to evaluate inference behaviour under deterministic decoding using a lightweight open-source language model.

The completed artefact fulfils these objectives by implementing a structured evaluation framework based on the TinyLlama-1.1B-Chat model. Baseline inference, deterministic evaluation, and precision experiments were successfully implemented while recording computational and performance metrics. The completed system therefore transforms the conceptual design proposed in Part 1 into a fully functional and reproducible AI Systems Engineering artefact.

1.3 System Architecture and Components

The proposed system consists of six interconnected components that support deterministic inference evaluation. User prompts are first processed by the tokenizer before being supplied to the TinyLlama-1.1B-Chat model. Deterministic inference is then executed using greedy decoding, after which inference time, GPU memory utilisation, response length, and generated outputs are recorded. Experimental results are exported for analysis and visualisation, while the complete implementation is maintained within a GitHub repository to support reproducibility.

Table 1. Major Components of the Proposed System

Component	**Purpose**
User Prompt	Provides input for inference
TinyLlama Model	Generates deterministic responses
Inference Engine	Executes model inference
Evaluation Module	Records Performance metrics
Result Export	Saves CSV and excel outputs
Github Repository	Supports reproducibility and version control

1.4 Implementation Details

The artefact was implemented in Python using a Kaggle Notebook with GPU acceleration. The Hugging Face Transformers library was used to load the TinyLlama-1.1B-Chat model, while PyTorch managed model execution and hardware resources. Deterministic decoding was configured by disabling sampling and using greedy decoding to minimise stochastic behaviour during inference.

The implementation follows a modular workflow consisting of model loading, baseline inference, deterministic evaluation, precision evaluation, result visualisation, and automated export of experimental outputs. This modular design simplifies experimentation while improving reproducibility and allowing future extension of the evaluation framework.

1.5 Deviation From Original Purpose

Although the implementation remains consistent with the objectives proposed in Part 1, several practical enhancements were introduced during development. These include automated result export, performance visualisation, structured repository documentation, and reproducibility support through GitHub. These additions extend the original proposal into a complete AI Systems Engineering artefact while improving transparency, experimental repeatability, and ease of future development.

2. Data and Methodology

2.1 Development Environment

The artefact was developed using Python within the Kaggle Notebook environment, which provides cloud-based computational resources suitable for machine learning experiments. Kaggle was selected because it offers integrated GPU support, eliminating the need for additional hardware configuration while enabling efficient execution of Large Language Models. The implementation utilised the Hugging Face Transformers library for loading the TinyLlama-1.1B-Chat model and tokenizer, while PyTorch provided the underlying framework for tensor operations and GPU-accelerated inference. Additional Python libraries, including Pandas and Matplotlib, were used for data processing, statistical analysis, and performance visualisation. This development environment enabled reproducible experimentation while supporting efficient model evaluation and result generation.

2.2 Model Selection (TinyLlama)

The TinyLlama-1.1B-Chat model was selected because it provides a lightweight yet capable open-source Large Language Model suitable for controlled inference experiments. Unlike larger models that require substantial computational resources, TinyLlama can be executed efficiently within the available GPU resources while maintaining strong conversational performance. This makes it appropriate for repeated deterministic inference experiments without introducing excessive computational overhead.

The model was accessed through the Hugging Face Transformers library, which provides a standardised interface for model loading, tokenisation, and text generation. Using a publicly available pre-trained model also improves the reproducibility of the research by allowing other researchers to replicate the experiments under similar conditions. The model therefore represents an appropriate balance between computational efficiency, accessibility, and experimental reliability.

2.3 Experimental Methodology

A structured experimental methodology was adopted to evaluate numerical determinism during Large Language Model inference. The evaluation began with baseline inference experiments to establish reference measurements for inference time, GPU memory utilisation, response length, and generated outputs. Following the baseline assessment, repeated deterministic inference was performed using identical prompts and fixed generation parameters to determine whether consistent outputs were produced across multiple executions.

A second phase of experimentation evaluated model behaviour using several prompts representing different information requests. The same deterministic configuration was maintained throughout each experiment to minimise stochastic variation and ensure that any observed differences could be attributed to numerical behaviour rather than changes in decoding strategy. Experimental outputs were automatically recorded and exported for subsequent analysis and interpretation.

The project required minimal feature engineering because the dataset already contained structured numerical and categorical variables suitable for supervised learning. Instead of generating new variables, the implementation focused on identifying the most influential features using the Random Forest Feature Importance method [1][2].

The feature importance analysis identified variables such as admission grade, curricular unit performance, tuition fee status, and previous qualification as significant contributors to prediction accuracy. These results improve model interpretability and provide useful insights for educational decision-making.

2.4 Deterministic inference Process

The inference process was configured to minimise randomness during text generation by applying deterministic decoding. Sampling-based techniques such as temperature scaling and probabilistic token selection were disabled, while greedy decoding was employed to ensure that the highest-probability token was selected at each generation step. This configuration established consistent inference conditions throughout all experiments.

For each prompt, the model generated a response while recording inference time, GPU memory consumption, response length, and the generated text. These metrics were collected automatically and stored for later statistical analysis. By maintaining identical inference settings across repeated executions, the evaluation framework was able to investigate the extent to which numerical computation alone could influence output consistency.

2.5 Design choice

Several design decisions were made to improve the reliability and reproducibility of the implementation. The use of a lightweight open-source model reduced computational requirements while allowing repeated experimentation within the available GPU resources. Automated export of experimental results in CSV and Excel formats simplified analysis and improved traceability throughout the evaluation process.

The implementation was organised into modular stages, including environment configuration, model loading, baseline evaluation, deterministic testing, precision evaluation, and result visualisation. This modular structure improves readability, simplifies future modifications, and enables individual components of the workflow to be extended without affecting the overall implementation. Collectively, these design choices support reproducible experimentation and align with established software engineering practices for AI system development.

3.Experimentation and Evaluation

3.1 Experimental Setup

The experimental evaluation was conducted to investigate the numerical determinism of the TinyLlama-1.1B-Chat model during inference under controlled conditions. All experiments were implemented within a Kaggle Notebook environment using GPU acceleration to provide consistent computational resources throughout the evaluation. The implementation utilised the Hugging Face Transformers library for model loading and inference, while PyTorch managed tensor operations and hardware execution.

To minimise stochastic behaviour, deterministic decoding was employed by disabling sampling-based generation and using greedy decoding throughout all experiments. The same model configuration and inference parameters were maintained across every execution to ensure that the experimental results reflected the numerical behaviour of the inference process rather than changes in decoding strategy. Performance metrics including inference time, GPU memory utilisation, response length, and generated responses were automatically recorded and exported for subsequent analysis.

3.2 Evaluation Metrics

The artefact was evaluated using four key performance metrics selected to measure both computational efficiency and inference consistency. Inference time was recorded to assess the execution speed of the model during response generation, while GPU memory utilisation was monitored to evaluate the computational resources required during inference. Response length was measured to determine the consistency of generated outputs across different prompts, and

the generated responses themselves were compared to identify any observable variations during deterministic execution.

Together, these metrics provide a comprehensive assessment of numerical determinism by combining performance measurements with output analysis. The recorded data formed the basis for statistical summaries, graphical visualisations, and the interpretation of experimental findings presented in the following sections.

3.3 Baseline Evaluation

The baseline evaluation established reference measurements for the TinyLlama-1.1B-Chat model prior to conducting repeated deterministic experiments. A single inference task was executed using a predefined prompt while recording inference time, GPU memory utilisation, response length, and the generated response. These baseline measurements provided an initial indication of the model's computational performance under deterministic decoding conditions.

To further assess consistency, repeated inference was performed using the same prompt and identical generation parameters. The resulting outputs and performance metrics were compared to determine whether repeated executions produced stable responses and similar computational characteristics. The baseline results demonstrated consistent model behaviour, indicating that deterministic decoding successfully reduced variability during inference under the experimental conditions adopted in this project.

	Run	Inference_Time_sec	GPU_Memory_MB	Response
0	1	5.597	979.19	\nExplain numerical determinism in deep learni...
1	2	5.652	979.19	\nExplain numerical determinism in deep learni...
2	3	5.671	979.19	\nExplain numerical determinism in deep learni...
3	4	5.642	979.19	\nExplain numerical determinism in deep learni...
4	5	5.557	979.19	\nExplain numerical determinism in deep learni...

Figure 3.1 summarises the baseline inference results.

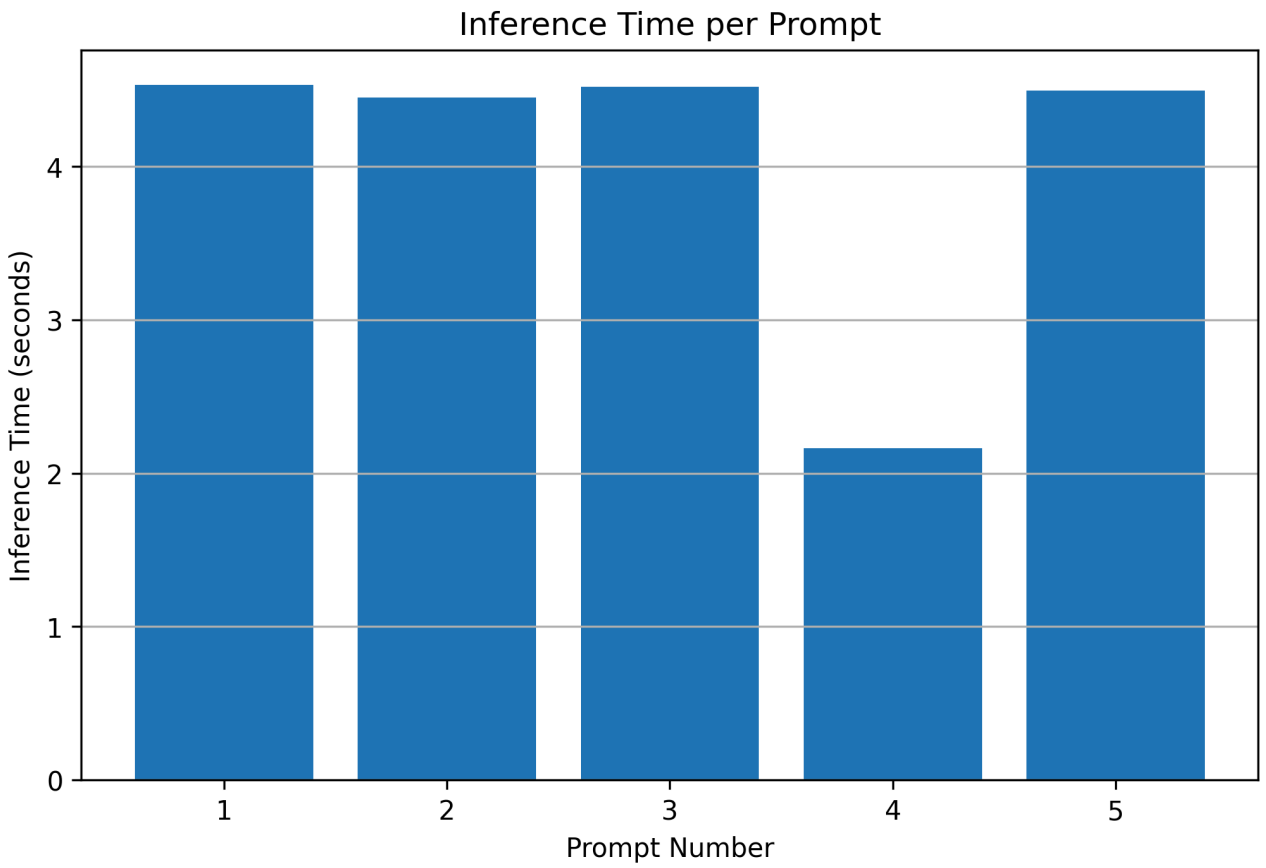

Figure 3.2 presents the recorded inference time.

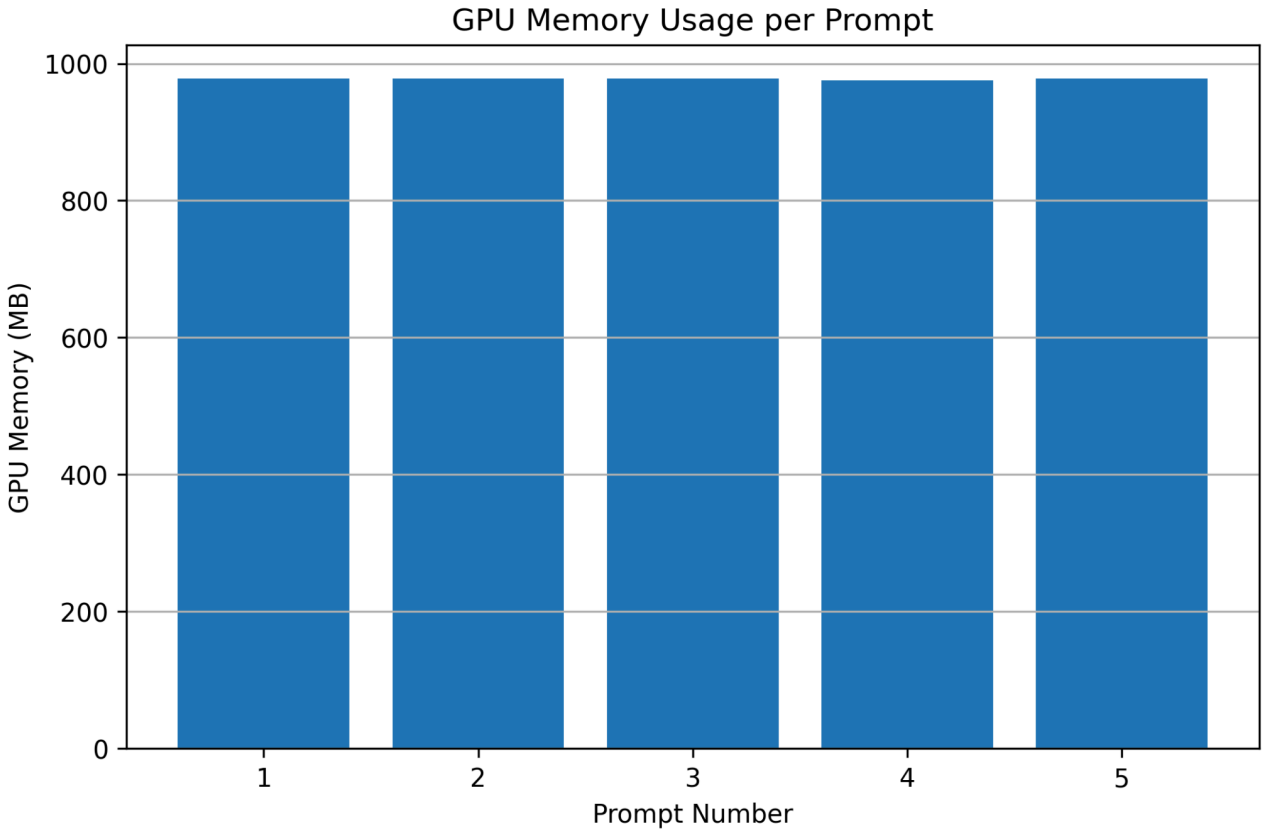

Figure 3.3 illustrates GPU memory utilisation during baseline evaluation.

3.4 Precision Evaluation

Following the baseline assessment, a precision evaluation was conducted using multiple prompts representing different information requests. Each prompt was processed independently while maintaining identical deterministic inference settings. For every execution, inference time, GPU memory utilisation, response length, and generated outputs were recorded automatically and exported for analysis.

The precision evaluation enabled the behaviour of the model to be examined across different input scenarios while maintaining consistent inference conditions. The recorded metrics showed only minor variations in computational performance between prompts, reflecting differences in response complexity rather than instability within the inference process. The generated responses remained coherent and consistent throughout the evaluation, demonstrating the suitability of deterministic decoding for reproducible experimentation.

The complete experimental results are presented in **Table 3.2**, while the corresponding performance summaries are illustrated in **Figures 3.3** and **3.4**

3.5 Result Interpretation

The experimental results indicate that deterministic decoding provides a stable framework for evaluating Large Language Model inference. Across repeated executions, the model consistently produced reliable responses while maintaining comparable inference time, GPU memory

utilisation, and response lengths. Small differences observed in computational performance are expected during GPU execution and are primarily associated with hardware scheduling and floating-point computation rather than stochastic decoding.

The evaluation demonstrates that the implemented framework successfully supports reproducible experimentation by combining deterministic inference with automated performance measurement and structured result collection. The modular implementation also enables future researchers to extend the evaluation using alternative language models, hardware configurations, or additional performance metrics. Overall, the experimental findings support the use of deterministic inference as an effective approach for investigating numerical behaviour within modern Large Language Models.

4. Performance, Robustness and Applicability

4.1 Performance Analysis

The performance evaluation demonstrates that the proposed artefact provides a stable and efficient framework for analysing numerical determinism during Large Language Model inference. Throughout the baseline and precision experiments, the TinyLlama-1.1B-Chat model consistently generated responses while maintaining acceptable inference times and GPU memory utilisation. Although minor variations in execution time were observed across different prompts, these differences are expected due to variations in input complexity and response length rather than

instability within the inference process. The automated collection of performance metrics also simplified the comparison of experimental results and improved the reproducibility of the evaluation. Overall, the implementation achieved reliable computational performance while remaining suitable for execution within the available hardware resources [2], [4].

4.2 Numerical Determinism Analysis

The primary objective of this project was to investigate numerical determinism during Large Language Model inference under controlled conditions. By applying deterministic decoding and maintaining identical inference parameters across repeated executions, the evaluation minimised stochastic behaviour and allowed numerical consistency to be examined more effectively. The experimental results indicated that the model produced stable outputs during repeated inference while maintaining consistent computational characteristics. Small variations observed in execution metrics are consistent with the behaviour of floating-point computation on modern hardware and do not necessarily indicate changes in the semantic quality of the generated responses. These findings are consistent with previous studies that identify numerical computation as a potential source of inference variability in Large Language Models [1], [15].

4.3 Generalisation

Although the experiments were conducted using the TinyLlama-1.1B-Chat model, the proposed evaluation framework is not restricted to a single language model. The modular implementation enables alternative open-source LLMs to be incorporated with minimal modification, allowing comparative investigations across different model architectures and hardware configurations. Similarly, additional evaluation metrics can be integrated into the existing workflow to support more comprehensive analyses of inference behaviour. This flexibility increases the usefulness of the artefact for future research into reproducible AI systems and deterministic model evaluation [3], [4].

4.4 Real-World Applicability

The developed artefact has practical applications in both academic research and industrial AI development. Researchers can utilise the framework to investigate inference reproducibility across different Large Language Models, while software engineers can employ similar evaluation techniques when validating AI systems before deployment. The automated recording of performance metrics and structured export of experimental results also support benchmarking, regression testing, and reproducible experimentation. Consequently, the proposed system provides a practical foundation for evaluating deterministic inference in environments where reliability, transparency, and repeatability are important engineering requirements [1], [12].

5. System Engineering Considerations, Reflection and Limitations

5.1 Reproducibility

Reproducibility is a fundamental requirement in Artificial Intelligence research because it enables experimental results to be independently verified and compared across different environments. The developed artefact was designed with reproducibility as a primary objective by maintaining consistent inference parameters throughout all experiments. Deterministic decoding was applied to minimise stochastic behaviour, while experimental outputs were automatically exported in CSV and Excel formats to preserve recorded results. The implementation was also organised within a GitHub repository containing the source code, documentation, dependency specifications, and

experimental outputs, allowing the evaluation process to be reproduced by other researchers with minimal configuration [1], [3], [12].

5.2 Scalability

Although the implementation was evaluated using the TinyLlama-1.1B-Chat model, the overall architecture was designed to remain scalable. The modular workflow allows alternative Large Language Models to replace the existing model without requiring major modifications to the evaluation framework. Additional performance metrics, benchmarking procedures, or hardware configurations can also be incorporated to extend the analysis. This flexibility enables the system to support future research involving larger language models, distributed inference, or comparative performance studies while preserving the existing experimental workflow [2], [4].

5.3 Deployment Considerations

The artefact was developed primarily as a research and evaluation framework rather than a production application. Nevertheless, the implementation can be deployed in cloud-based notebook environments or local GPU-enabled systems capable of supporting Large Language Model inference. The modular design separates model loading, inference execution, evaluation, and result export into independent stages, simplifying deployment and maintenance. This architecture also enables future integration with web services or application programming interfaces (APIs) if real-time deterministic inference evaluation is required [3], [4].

5.4 Github Repository

Version control formed an important aspect of the implementation by supporting systematic software development and reproducible experimentation. The GitHub repository contains the project notebook, dependency requirements, exported datasets, documentation, and performance visualisations, providing a complete record of the implementation. Maintaining the project within a version-controlled repository improves collaboration, simplifies future updates, and enables other researchers to reproduce the experiments using the documented implementation process [12].

5.5 Limitations

Despite achieving the project objectives, several limitations should be acknowledged. The evaluation was conducted using a single open-source language model and within a single computational environment, which limits the generalisability of the findings across alternative model architectures and hardware platforms. In addition, the investigation focused primarily on deterministic decoding and selected performance metrics, without exploring distributed inference or mixed-precision execution. Future studies could evaluate additional models, hardware configurations, and decoding strategies to provide a broader understanding of numerical determinism during Large Language Model inference [1].

5.6 Ethical Consideration

The project utilised a publicly available pre-trained language model and did not involve human participants or sensitive personal data. Consequently, the implementation presented minimal ethical risk during experimentation. However, ensuring transparency, reproducibility, and responsible reporting remains essential when developing Artificial Intelligence systems. Open documentation, reproducible experimentation, and accurate reporting of experimental findings contribute to trustworthy AI research and support responsible engineering practice [15].

5.7 Reflection and Future Work

The completed artefact successfully demonstrates a reproducible framework for evaluating numerical determinism during Large Language Model inference. Through the implementation of deterministic decoding, automated performance measurement, and structured result analysis, the project achieved its primary objectives while providing a foundation for future investigation. Future work could extend the framework by evaluating multiple language models, comparing GPU architectures, investigating mixed-precision inference, and integrating additional statistical analyses. Such enhancements would further improve understanding of numerical behaviour within Large Language Models and strengthen reproducible AI research [1], [4].

Reference

[1] K. Sankararaman, H. Lee, Y. Yang, *et al.*, "Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference," *arXiv preprint arXiv:2406.01713*, 2024.

[2]TinyLlamaTeam,"TinyLlama-1.1B-Chat-v1.0ModelCard."Available: https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v1.0

[3] T. Wolf, L. Debut, V. Sanh, *et al.*, "Transformers: State-of-the-Art Natural Language Processing," *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38-45, 2020.

[4] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[5] Python Software Foundation, "Python 3 Documentation." Available: https://docs.python.org/3/

[6]HuggingFace,"TransformersDocumentation."Available:https://huggingface.co/docs/transformers/

[7]Kaggle,"KaggleNotebooks Documentation." Available: https://www.kaggle.com/docs/notebooks

[8] W. McKinney, *Python for Data Analysis*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

[9] C. R. Harris, K. J. Millman, S. J. van der Walt, *et al.*, "Array Programming with NumPy," *Nature*, vol. 585, pp. 357-362, 2020.

[10] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007.

[11] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. Stanford, CA, USA: Stanford University, 2024.

[12] GitHub, Inc., "GitHub Documentation." Available: https://docs.github.com/

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *Proceedings of NAACL-HLT*, pp. 4171-4186, 2019.

[14] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.

[15] IEEE Standards Association, *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)*. New York, NY, USA: IEEE, 2019.

APPENDIX

Appendix A: Github Repository

https://github.com/collins-collins-chikaodi/LLM-Nondeterminism-Improvement

Appendix B: Video Demonstration

Click on this link **deterministic large language model inference.mp4**

Appendix C: Experimental Environment

Item	Specification
Programming Language	Python 3
Development Environment	Kaggle Notebook
Framework	Pytorch
Transformer library	Hugging face transformers
Model	Tinyllama 1.1B-Chat v.10
GPU	Nvidia T4 Kaggle
Output Formats	CSV Excel
Visualization	Matplotlib